arXiv:2506.23529v1 [cs.CV] 30 Jun 2025

When Test-Time Adaptation Meets Self-Supervised Models

Jisu Han[1*] Jihee Park[1*] Dongyoon Han[2] Wonjun Hwang[1†]
[1]Korea University [2]NAVER AI Lab

Abstract

Training on test-time data enables deep learning models to adapt to dynamic environmental changes, enhancing their practical applicability. Online adaptation from source to target domains is promising but it remains highly reliant on the performance of source pretrained model. In this paper, we investigate whether test-time adaptation (TTA) methods can continuously improve models trained via self-supervised learning (SSL) without relying on source pretraining. We introduce a self-supervised TTA protocol after observing that existing TTA approaches struggle when directly applied to self-supervised models with low accuracy on the source domain. Furthermore, we propose a collaborative learning framework that integrates SSL and TTA models, leveraging contrastive learning and knowledge distillation for stepwise representation refinement. We validate our method on diverse self-supervised models, including DINO, MoCo, and iBOT, across TTA benchmarks. Extensive experiments validate the effectiveness of our approach in SSL, showing that it achieves competitive performance even without source pretraining.

1. Introduction

Deep neural networks (DNNs) have achieved remarkable advancements across various fields [4, 9, 13, 36] of computer vision and are increasingly becoming a standard tool in the industry [20, 44, 50]. However, the issue of performance degradation due to domain shift [38] between training and test datasets remains an unresolved challenge, even when distributional differences appear to be minimal [34]. To address this challenge, Test-Time Training (TTT) introduces a new paradigm in domain adaptation by training at test-time to address distributional shifts between training and test data [10, 25, 40]. Building on the principles of TTT, various protocols have been developed to extend its practicality. Test-Time Adaptation (TTA) further extends this idea by adapting a pretrained model to the test domain without requiring access to source data, addressing concerns related to privacy and memory constraints [22, 28, 45, 54], and Continual Test-Time Adaptation (CTTA) extends TTA by assuming a continuously evolving test distribution, where the model adapts sequentially over time [1, 12, 24, 47].

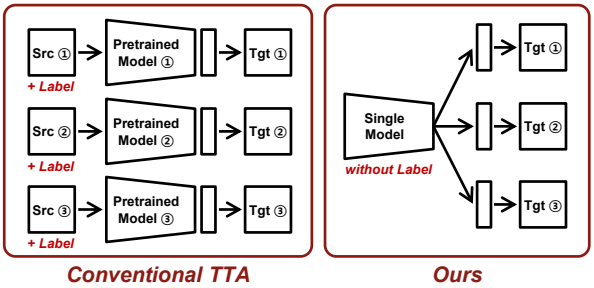


Figure 1. **Concept of Self-Supervised TTA.** Conventional TTA methods require a separate pretraining for each source domain, whereas our Self-Supervised TTA eliminates the need for source-specific pretraining by leveraging self-supervised learning.

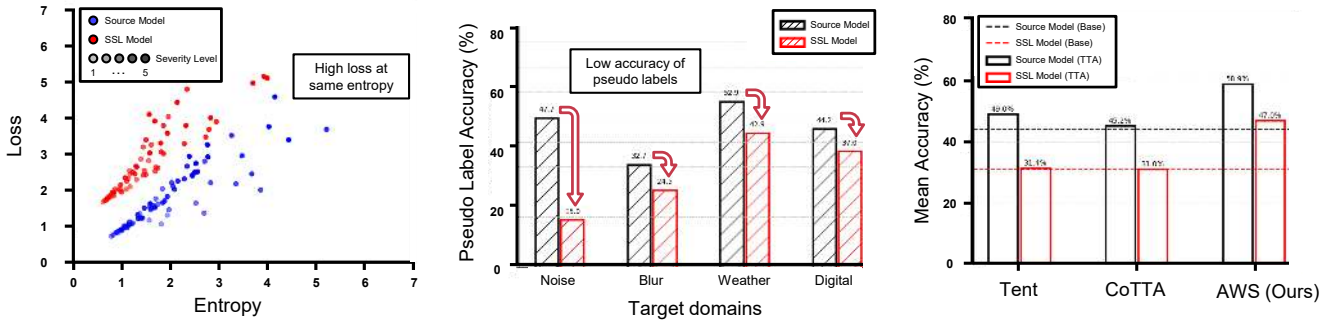
Source Model	ImageNet	CIFAR100
Source Pretraining	1h8m23s×300epochs	9m7s×200epochs
SSL w/ Prototype	36m25s	1m25s
SSL w/ Prototype (Few-Shot)	1m56s	7s

Table 1. **Training time comparison** between the source pretraining of the conventional TTA and our approach.

Despite many achievements of TTA, discussions on the pretraining model prepared using source data and corresponding labels have been limited. For example, as shown in Fig. 1, conventional TTA required a pretraining model trained on CIFAR10 [21] to adapt to CIFAR10C (i.e., corruption set), but this model did not perform well on CIFAR100C. In other words, a separate pretraining model had to be prepared for each target domain. This limitation poses challenges in terms of practical efficiency and generality.

Along with this, our study began with a simple question: "*Is the computational cost of pretraining the source model negligible compared to the adaptation process for unlabeled target data in TTA?*" We unveil the training time required for TTA methods using a pretrained source model in Tab. 1, shedding light on the overlooked cost of source domain training and bringing it into the discussion. Opti-

*Equal contribution.
†Corresponding author.



(a) Failure of the EM Method in SSL model (b) Failure of the CR Method in SSL model (c) Performance comparison in SSL model

Figure 2. **Analysis of Self-Supervised models in Test-Time Adaptation.** (a) The relationship between entropy and loss for source pretrained and SSL models. SSL models tend to exhibit higher loss for the same entropy level and may decrease the entropy of incorrect predictions, thereby increasing the true risk. (b) The accuracy of pseudo-labels for different target domains. SSL models generate pseudo labels with lower accuracy compared to source pre-trained models, which hinders performance improvement due to the propagation of inaccurate supervision signals. (c) Comparison of accuracy across different TTA approaches. Our AWS achieves improved performance for the SSL model compared with EM [45] and CR [47] methods.

mizing the pretraining process of the source model is a practical matter, especially considering that labeled source data is often unavailable or prohibitively expensive to obtain. A simple solution is to leverage the zero-shot performance of a self-supervised model trained through Self-Supervised Learning (SSL) on large-scale datasets [2, 6, 7, 31, 57]. This approach enhances generalization without requiring explicit supervision from the source domain, thereby mitigating the computational burden associated with pretraining while maintaining competitive adaptation performance in target domains. Specifically, we improve computational efficiency by designing a distance-based classifier that utilizes class prototypes obtained only through forward passes.

In this paper, we conduct an empirical investigation into the effectiveness of existing TTA approaches on self-supervised models without domain-specific knowledge and explore the feasibility of applying SSL for TTA. Figs. 2a and 2b show that the primary TTA approaches, Entropy Minimization (EM) [45] and Consistency Regularization (CR) [47], are not readily applicable to SSL models. EM method minimizes predictive entropy based on the observation that lower entropy indicates higher model accuracy. While it has been demonstrated to be effective for conventional TTA, its applicability remains challenging in SSL models, where low entropy does not ensure accurate predictions. Furthermore, CR approaches that leverage pseudo-labels to maintain predictive consistency also suffer from the inaccuracy of pseudo-labels based on the low domain accuracy of SSL models.

Given that the SSL model does not seamlessly extend to TTA, we introduce a novel framework called Adapt Without Source pretraining (AWS). The proposed method consists of three key components. First, contrastive learning enhances the representation capability for both source and target domains. Second, knowledge distillation preserves the generalization ability of the initial SSL model. Third, mutual learning integrates the advantages of different predictions from the SSL and target models. Fig. 2c presents the TTA performance of a source model trained with supervised learning on the source domain and a self-supervised model, DINO [2]. Compared to EM and CR approaches, which fail to enhance the performance of SSL models, our method demonstrates its effectiveness in improving TTA performance for SSL models. Notably, despite the initial performance gap on the target domain, our approach surpasses the source-pretrained model, highlighting the potential for advancing TTA using SSL models. In summary, the main contributions of our work are as follows:

- To the best of our knowledge, we are the first to highlight the issue of computational efficiency in the source training process for TTA. Motivated by this challenge, we propose a self-supervised test-time adaptation protocol.
- We investigate the potential of using self-supervised models in TTA and identify failure cases through an empirical analysis of prominent methodologies in TTA
- Beyond conventional TTA approaches that rely on the performance of a source model, we propose AWS, a novel method consisting of contrastive learning, knowledge distillation, and mutual learning. Our approach is extensively evaluated under standard TTA protocols and across various SSL models, demonstrating its effectiveness.

2. Related Work

2.1. Test-Time Adaptation

Distributional discrepancies between the source and target domains present a significant challenge during the deployment of DNNs [38], and TTT introduces a learning approach that operates during test time [40]. TTT mitigates domain shift by adopting supervised learning on the

Setting	Pretrain Loss	Source Data	Target Data	Data Distribution	Train Loss	Test Loss
Fine-Tuning	-	-	x^t, y^t	-	$L(x^t, y^t)$	-
Domain Adaptation	-	x^s, y^s	x^t	-	$L(x^s, y^s) + L(x^t, x^s)$	-
Source-Free Domain Adaptation	$L(x^s, y^s)$	-	x^t	-	$L(x^t)$	-
Test-Time Training	-	x^s, y^s	x^t	stationary	$L(x^s, y^s) + L(x^t)$	$L(x^t)$
Fully Test-Time Adaptation	$L(x^s, y^s)$	-	x^t	stationary	-	$L(x^t)$
Continual Test-Time Adaptation	$L(x^s, y^s)$	-	x^t	continually changing	-	$L(x^t)$
Self-Supervised Test-Time Adaptation	$L(x^u)$	-	x^t	continually changing	-	$L(x^t)$

Table 2. **Comparison of different adaptation protocols.** Existing protocols require training on source data during the adaptation or pretraining process. Self-Supervised Test-Time Adaptation is based on unlabeled data x^u, which is not the source domain, and does not involve training on the source data. For source domain, only a forward pass over full or few-shot is performed, without backpropagation.

source domain and self-training on unlabeled target domain data [10, 25, 32]. In contrast, TTA emphasizes the impracticality of accessing source domain data and instead proposes an adaptation strategy that is solely applied at test time using a source pretrained model [45]. The main solution for TTA is the EM-based approach [22, 27, 28, 55]. The EM approach updates only the normalization layer and filters out inaccurate samples from the observation that samples with low entropy perform relatively well. Moreover, CTTA proposes a solution to address scenarios involving continuous domain shifts [47]. CR is a primary solution in CTTA and has gained prominence for its effectiveness in stabilizing adaptation over time [1, 23, 24, 47]. The CR approach utilizes a teacher-student framework [41] that updates all model parameters, enabling gradual adaptation through Exponential Moving Average (EMA) update. By leveraging pseudo labels generated by an augmented teacher model, CR enforces consistency throughout the adaptation process.

2.2. Self-Supervised Learning

The training of increasingly deeper and more complex DNNs demands large amounts of data. However, the expensive cost of human annotation presents challenges for supervised learning. SSL has been proposed as an alternative, leveraging unlabeled data for various downstream tasks [2, 5, 6, 14, 30, 31, 57]. CPC [30] introduces a representation learning approach based on probabilistic contrastive learning for future prediction. MoCo [14] employs a memory bank and a momentum encoder to facilitate contrastive learning with a large and consistent set of negative samples. SimCLR [5] leverages strong data augmentations and a contrastive loss to maximize similarity between augmented views of the same instance. DINO [2] adopts a self-distillation and teacher-student framework with a momentum encoder. iBOT [57] proposes an mask prediction-based SSL framework through masked image modeling.

In this paper, we empirically investigate the effectiveness of TTA strategies in practical scenarios where labels are unavailable during the source pretraining phase. Furthermore, we propose Self-Supervised TTA, which leverages an SSL model as the source model and integrates it into the TTA.

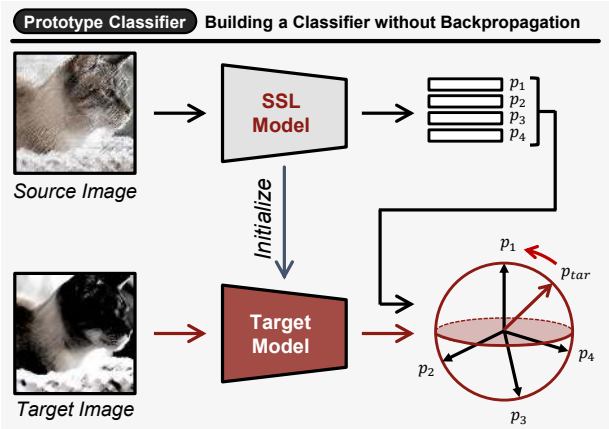


Figure 3. **A framework without source pretraining.** We construct a prototype classifier only through forward passes without a training process on the source domain.

3. Self-Supervised Test-Time Adaptation

3.1. Protocol

We briefly summarize the well-known adaption protocols for simple comparison in Tab. 2, including the method replacing the source pre-training process in Fig. 3 and the overview of our method is also illustrated in Fig. 4.

Source Model. Conventional TTA protocols [23, 24, 28, 45, 47, 54] based on supervised learning of a source model $g_s \circ f_s$ using labeled source domain data $(x^s, y^s) \in \{\mathcal{X}^s, \mathcal{Y}^s\}$, where g_s and f_s represent the classifier and feature extractor of the source model, respectively. Instead of requiring pretraining on the source domain, we employ a self-supervised model f_{ssl} trained on an unlabeled data $x^u \in \mathcal{X}^u$. We compute feature prototypes from either a subset or the entire source dataset to align the representation of the SSL model with each class and construct a classifier g_{ssl}. Further details on the g_{ssl} are provided in Sec. 3.2.

Target Adaptation. We follow the CTTA protocol [47], which assumes a continuously changing environment without explicit domain boundaries, to assess the adaptability of the SSL model to the target domain. The target model $g_t \circ f_t$ is initialized from the SSL model $g_{\text{ssl}} \circ f_{\text{ssl}}$. Our main objec-

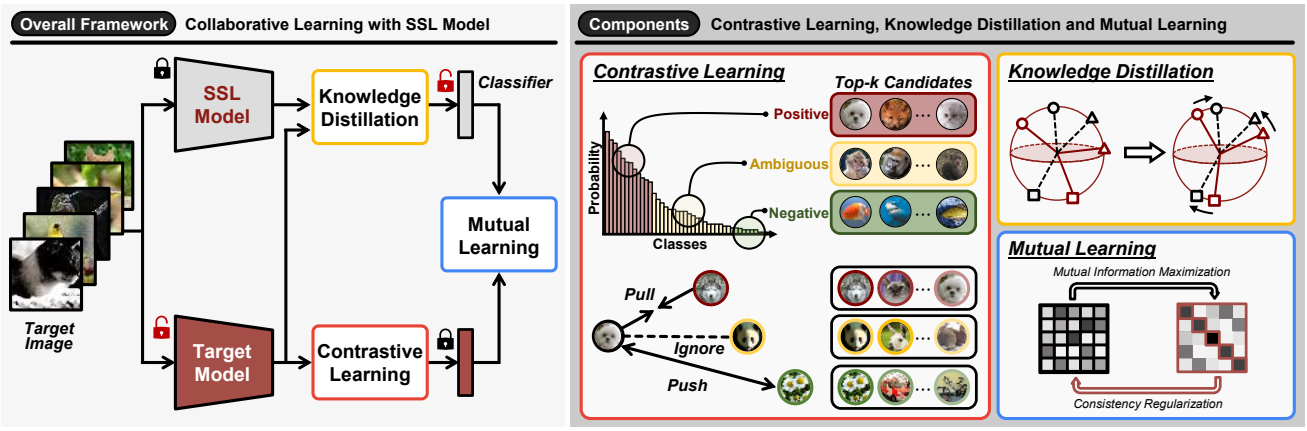


Figure 4. **Overview of our AWS framework.** Contrastive learning refines representations by leveraging pseudo-labels while maintaining stability, knowledge distillation preserves generalization by aligning feature representations to mitigate overfitting under domain shifts, and mutual learning improves adaptation by integrating the generalization ability of the SSL model with the domain-specific knowledge of the target model through pseudo-labeling.

tive is to adapt to the target domain by leveraging an online stream of unlabeled target data $x^t \in \mathcal{X}^t$ while minimizing the mean error as the domain gradually shifts.

3.2. Methodology

Prototype Classifier. The primary challenge in applying the SSL model to TTA is the absence of a classifier corresponding to each class. Linear probing and the k-nearest neighbor (k-NN) classifier are widely used methods for building a classifier that aligns with each class [5, 14, 30]. However, linear probing necessitates backpropagation for gradient computation, whereas the k-NN classifier entails substantial computational and memory overhead due to the requirement of storing a large number of feature representations. Inspired by the prototype-based classification in few-shot learning [26, 39] and continual learning [18, 33], we establish a prototype μ_c for each class c and employ a cosine similarity-based classifier. Using only the forward pass enhances computational efficiency. The prediction probability for each class is given by

$$p_t(y = c|x) = \frac{\exp(\sigma \cdot \cos(f_t(x), \mu_c))}{\sum_{i \in C} \exp(\sigma \cdot \cos(f_t(x), \mu_i))}, \quad (1)$$

where $cos(\cdot, \cdot)$ denotes the cosine similarity between two vectors, σ represents the logit scaling factor, C denotes the total number of classes and μ_c is the mean of features for each class c for the source dataset $\{\mathcal{X}^s, \mathcal{Y}^s\}$ of the SSL model, i.e., $\mu_c = \frac{1}{|\mathcal{X}^s_c|} \sum_{\mathcal{Y}^s_c} f_{ssl}(x^s)$.

Contrastive Learning. Through a contrastive loss function, distance-based classifiers benefit from improved performance while enabling the gradual refinement of representations [3, 5, 30, 49]. Building on the need for robustness against uncertainty induced by domain shifts, we introduce an approximately correct contrastive learning method that integrates a refined segmentation of multiple prediction candidates [53]. Compensating for the low accuracy in the target domain, we identify samples sharing a pseudo label $\mathcal{T}^k$ within the top-k predictions as positive samples. Conversely, when no common prediction exists among $\mathcal{T}^n$, which denotes the top-n predictions with $n > k$, the sample is treated as a negative instance. For ambiguous samples that do not fit either category, contrastive loss is not applied. Accordingly, the indicator function is defined as

$$\mathbb{1}_{ij} = \begin{cases} 1, & \text{if } \mathcal{T}^k_i \cap \mathcal{T}^k_j \neq \emptyset \\ -1, & \text{if } \mathcal{T}^n_i \cap \mathcal{T}^n_j = \emptyset \ (n > k) \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

We estimate the relationships among samples predicted as positive, ambiguous, or negative using the indicator function. By applying contrastive loss to these approximately correct sample relationships, we actively leverage the initial classification capability of the SSL model while ensuring stability. The approximately correct contrastive learning loss is defined as follows:

$$\mathcal{L}_{cl} = -\sum_{i=1}^{B} \sum_{j=1}^{B} \frac{\mathbb{1}_{ij}}{\sum_{j=1}^{B} \mathbb{1}_{ij}} \log \frac{\exp(S^t_{ij})}{\sum_{k=1}^{B} \exp(S^t_{ik})}, \quad (3)$$

where S^t_{ij} represents the cosine similarity between $f_t(x_i)$ and $f_t(x_j)$, and B denotes the batch size.

Knowledge Distillation. As a fundamental technique for transferring knowledge between models, knowledge distillation [17] has demonstrated effectiveness in various domains, including model compression [37, 52], mitigating

catastrophic forgetting [18, 33], improving zero-shot performance [43, 56]. To preserve generalization performance and mitigate overfitting under continuous domain shifts, we transfer knowledge from the SSL model to the target model. By reducing the rotation of feature representations, we retain the knowledge embedded in the SSL model while ensuring prediction consistency in the prototype classifier, which relies on cosine similarity between feature vectors and weight vectors of the classifier. To this end, we propose a knowledge distillation loss that aligns normalized feature vectors, facilitating stable knowledge transfer while preserving the geometric structure of the feature space.

$$\mathcal{L}_{kd} = \frac{1}{B}\sum_{i=1}^{B} \|\overline{f}_t(x_i) - \overline{f}_{ssl}(x_i)\|_2, \tag{4}$$

where $\overline{f}(x) = \frac{f(x)}{\|f(x)\|}$ denotes normalized feature vector, and $\|\cdot\|_2$ represents the Frobenius norm.

Mutual Learning. A self-supervised model demonstrates generalization performance by training on large-scale datasets, whereas a target model acquires domain-specific knowledge through adaptation. Drawing insight from studies suggesting that collaborative learning between models enhances robustness to noisy labels [11, 48, 51], we propose a collaborative mutual learning framework to integrate the strengths of these distinct predictive tendencies. To adapt the model to the target domain, we update the SSL model's classifier using pseudo labels generated by the target model, which maintains relatively high accuracy. This enables classifier refinement while preserving the fixed embeddings of the SSL model. Furthermore, we maximize the mutual information between predicted probability distributions to capture relational information between samples, leveraging the SSL model's representational capabilities. The collaborative loss for mutual knowledge transfer is formulated as follows:

$$\mathcal{L}_{ml} = \frac{1}{B}\sum_{i=1}^{B} [\underbrace{\mathcal{H}(p_i^{ssl}, \hat{p}_i^t)}_{\text{loss for SSL}} + \underbrace{I(p_i^t, p_i^{ssl})}_{\text{loss for target}}], \tag{5}$$

where p_i^t denotes the probability obtained by applying the softmax function to $g_t \circ f_t(x_i)$ and $\hat{p}_i^t = \text{argmax}(p_i^t)$. $I(p, q)$ represents the mutual information [19], and $\mathcal{H}(p, q)$ is cross entropy between two probability distributions p and q.

The total loss function of the proposed method, which consists of approximately correct contrastive learning, knowledge distillation, and mutual learning, is formulated as follows:

$$\mathcal{L}_{aws} = \mathcal{L}_{cl} + \lambda_{kd}\mathcal{L}_{kd} + \lambda_{ml}\mathcal{L}_{ml}, \tag{6}$$

where λ_{kd} and λ_{ml} are hyperparameters for knowledge distillation loss and mutual loss, respectively.

4. Experiments

In this section, we begin by evaluating the mean error on the target domain using a source pretrained model, which is pretrained on the source domain [8, 21, 34] in a supervised manner. We then apply this evaluation to our proposed self-supervised TTA protocol, utilizing DINO [2], MoCo [6], and iBOT [57] to measure the mean error. Sec. 4.2 describes the results for the source pretrained models, and Sec. 4.3 for the self-supervised models.

4.1. Experimental setup

Datasets and Models. We conduct our experiments on standard CTTA benchmarks, including ImageNet-to-ImageNetC [15], CIFAR10-to-CIFAR10C, and CIFAR100-to-CIFAR100C [21]. ImageNetC, CIFAR10C, and CIFAR100C are corruption sets for each source data, with 15 types of 4 main categories, which serve as sequential target domains. Following [23, 24, 47], we sequentially adapt the pretrained model to 15 target domains with the highest corruption level of 5 and evaluate its online prediction performance by measuring the mean error rate. Following [23, 24], we adopt ViT-B/16 [9] as the backbone network. We present experimental results for both source pretrained and self-supervised models, using DINO [2], MoCo [6], and iBOT [57] as SSL models.

Compared Methods. We compare our AWS with the well-known state-of-the-art methods: Tent [45], SAR [28], COME [55], CoTTA [47], PETAL [1], ViDA [24], and Continual-MAE [23]. ViDA and Continual-MAE require additional training as they incorporate an extra adapter into the source model. This makes it challenging to apply them using self-supervised models. Therefore, we do not include their results on self-supervised models.

Implementation Details. We employ the SGD optimizer with a momentum of 0.9 for training on the target domain. The batch size is 64 for ImageNetC and 16 for CIFAR datasets. The learning rate is set to $1e$-$4\times\frac{\text{batch size}}{64}$ for the source pretrained models, and we select the range of [$1e$-3, $1e$-4, $1e$-5, $1e$-6]$\times\frac{\text{batch size}}{64}$ for the self-supervised models. More implementation details in Appendix A.

4.2. Results on Source Pretrained Models

ImageNet-to-ImageNetC. Tab. 3 shows the error rate for each corruption, mean error, and the corresponding performance gain in the ImageNet-to-ImageNetC scenario. The pretrained model column indicates the source model used in the experiment and the accuracy for the source domain is under each model. As shown in the source-pretrained section, we achieve the best performance of 39.4%, surpassing the prior state-of-the-art method, Continual-MAE.

CIFAR10-to-CIFAR10C & CIFAR100-to-CIFAR100C. For the CIFAR datasets, we summarize the mean error rates in Tab. 4. We achieve error rates of 10.8% on CIFAR10C

Time		t →																
Pretrained Model	Method	Gaussian	shot	impulse	defocus	glass	motion	zoom	snow	frost	fog	brightness	contrast	elastic_trans	pixelate	jpeg	Mean↓	Gain↑
Source pretrained Acc. 83.6%	No Adapt [9] [Baseline]	53.0	51.8	52.1	68.5	78.8	58.5	63.3	49.9	54.2	57.7	26.4	91.4	57.5	38.0	36.2	55.8	0.0
	Tent [45] [ICLR'21]	52.2	48.9	49.2	65.8	73.0	54.5	58.4	44.0	47.7	50.3	23.9	72.8	55.7	34.4	33.9	51.0	+4.8
	CoTTA [47] [CVPR'22]	52.9	51.6	51.4	68.3	78.1	57.1	62.0	48.2	52.7	55.3	25.9	90.0	56.4	36.4	35.2	54.8	+1.0
	SAR [28] [ICLR'23]	49.3	43.8	44.9	58.2	60.9	46.1	51.8	41.3	44.1	41.8	23.8	57.2	49.9	32.9	32.7	45.2	+10.6
	PETAL [1] [CVPR'23]	52.1	48.2	47.5	66.8	74.0	56.7	59.7	46.8	47.2	52.7	26.4	91.3	50.7	32.3	32.0	52.3	+3.5
	ViDA [24] [ICLR'24]	47.7	42.5	42.9	52.2	56.9	45.5	48.9	38.9	42.7	40.7	24.3	52.8	49.1	33.5	33.1	43.4	+12.4
	Continual-MAE [23] [CVPR'24]	46.3	41.9	42.5	51.4	54.9	43.3	**40.7**	**34.2**	**35.8**	64.3	**23.4**	60.3	**37.5**	**29.2**	31.4	42.5	+13.3
	COME [55] [ICLR'25]	49.3	43.5	44.5	59.6	60.1	49.4	52.4	41.6	43.6	44.3	24.1	89.1	45.9	32.4	32.5	47.5	+8.3
	AWS [Proposed]	**43.9**	**39.6**	**41.3**	**48.9**	**47.7**	**42.2**	42.9	35.8	37.3	**39.7**	23.6	**49.8**	**37.5**	30.9	**30.3**	**39.4**	**+16.4**
DINO [2] Acc. 63.1%	No Adapt [9] [Baseline]	85.7	83.6	85.7	68.7	86.5	73.3	73.4	64.3	64.3	61.8	38.1	79.8	65.7	55.8	50.8	69.2	0.0
	Tent [45] [ICLR'21]	81.8	75.9	75.6	67.3	94.0	73.6	73.4	62.1	62.7	61.4	38.2	75.4	67.9	51.9	48.6	67.3	+1.9
	CoTTA [47] [CVPR'22]	98.2	99.1	99.3	68.7	78.7	72.0	70.9	69.9	64.9	61.7	41.0	78.1	59.8	52.9	51.8	71.1	-1.9
	SAR [28] [ICLR'23]	81.0	73.5	73.3	68.8	91.0	73.0	72.1	61.8	62.5	61.1	38.2	74.6	67.6	51.7	48.5	66.6	+2.6
	PETAL [1] [CVPR'23]	97.8	98.1	98.5	68.0	86.6	74.7	72.8	64.6	64.6	60.7	38.3	80.2	66.5	55.6	51.2	71.9	-2.7
	COME [55] [ICLR'25]	85.7	83.5	85.7	68.6	86.5	73.3	73.4	64.2	64.2	61.6	38.1	80.3	65.7	56.5	51.2	69.2	+0.0
	AWS [Proposed]	**65.9**	**59.6**	**60.7**	**57.8**	**59.3**	**57.0**	**52.7**	**50.8**	**50.9**	**50.3**	**37.0**	**52.6**	**49.6**	**45.0**	**45.6**	**53.0**	**+16.2**
	AWS-FS [Proposed]	66.7	61.0	63.0	59.1	61.5	57.9	53.5	52.3	52.1	51.2	39.1	54.3	50.7	46.3	47.7	54.4	+14.8
MoCo [6] Acc. 60.0%	No Adapt [9] [Baseline]	91.2	89.5	92.1	79.9	90.2	79.8	82.6	74.3	76.4	80.3	43.1	85.4	71.2	52.6	59.6	76.5	0.0
	Tent [45] [ICLR'21]	91.2	89.5	92.1	79.9	90.2	79.8	82.7	74.3	76.4	80.4	43.1	85.4	71.2	52.7	59.7	76.6	-0.1
	CoTTA [47] [CVPR'22]	96.9	94.3	98.1	80.8	95.6	82.7	83.8	74.6	76.1	78.1	42.9	86.7	70.9	52.1	59.0	78.2	-1.7
	SAR [28] [ICLR'23]	91.1	89.1	91.2	79.9	90.7	78.7	82.0	72.6	73.7	78.0	41.6	85.4	68.8	51.0	57.2	75.4	+1.1
	PETAL [1] [CVPR'23]	96.9	94.3	98.1	80.8	95.6	82.7	83.9	74.8	76.2	77.8	42.9	86.4	71.1	51.9	59.2	78.2	-1.7
	COME [55] [ICLR'25]	91.1	89.1	91.1	79.9	90.8	78.7	81.9	72.6	73.0	77.1	41.3	85.2	68.7	51.3	57.5	75.3	+1.2
	AWS [Proposed]	90.2	84.6	83.7	**74.2**	81.3	**73.7**	**74.9**	**66.1**	**65.7**	**68.7**	**44.5**	**73.0**	**62.2**	**48.8**	**51.6**	**69.5**	**+7.0**
	AWS-FS [Proposed]	**89.4**	**81.9**	**81.4**	79.9	**80.5**	80.1	80.1	74.3	70.6	78.8	52.7	78.5	66.7	56.4	56.5	73.8	+2.7
iBOT [57] Acc. 65.9%	No Adapt [9] [Baseline]	86.1	84.2	86.9	69.3	87.6	74.6	73.3	62.3	62.5	60.3	36.1	78.5	62.2	48.9	47.2	68.0	0.0
	Tent [45] [ICLR'21]	86.1	84.0	87.2	68.8	88.4	71.3	71.2	60.5	61.3	60.3	36.3	79.4	63.2	47.1	48.0	67.5	+0.5
	CoTTA [47] [CVPR'22]	86.1	84.3	87.0	69.3	87.6	77.3	73.3	61.8	61.9	60.0	36.1	78.0	61.9	48.4	46.7	68.0	+0.0
	SAR [28] [ICLR'23]	85.7	83.2	85.1	68.8	87.9	70.9	71.3	60.0	61.1	60.3	36.2	78.3	62.7	47.1	47.7	67.1	+0.9
	PETAL [1] [CVPR'23]	86.1	84.3	87.0	69.3	87.6	77.3	73.3	61.6	61.8	59.9	36.0	77.9	61.9	48.3	46.7	67.9	+0.1
	COME [55] [ICLR'25]	86.2	84.2	87.0	69.2	87.6	74.5	73.3	62.4	62.5	60.3	36.2	78.4	66.2	48.9	47.1	68.0	+0.0
	AWS [Proposed]	**56.4**	**51.5**	**53.4**	**53.3**	**55.0**	**52.5**	**48.5**	**46.3**	**48.1**	**46.6**	**34.8**	**47.4**	**44.6**	**40.5**	**42.8**	**48.1**	**+19.9**
	AWS-FS [Proposed]	58.2	53.3	55.2	55.6	56.0	54.3	50.8	48.7	49.7	48.4	36.4	49.8	45.8	42.3	44.3	49.9	+18.1

Table 3. **Classification error rate (%) for ImageNet-to-ImageNetC under CTTA scenario**. Mean (%) denotes the average error rate across 15 target domains. Gain (%) represents the improvement over the baseline. FS denotes the few-shot setup that utilizes a prototype classifier constructed with 30 samples per class. The **bold** indicates best performance.

Pretrained Model	Source Pretrained		DINO [2]		MoCo [6]		iBOT [57]	
Method	CIFAR10C	CIFAR100C	CIFAR10C	CIFAR100C	CIFAR10C	CIFAR100C	CIFAR10C	CIFAR100C
No Adapt [9] [Baseline]	28.2 (0.0)	35.4 (0.0)	44.3 (0.0)	64.1 (0.0)	42.2 (0.0)	64.2 (0.0)	48.0 (0.0)	65.6 (+0.0)
Tent [45] [ICLR'21]	23.5 (+4.7)	32.1 (+3.3)	43.5 (+0.8)	62.9 (+1.2)	42.7 (-0.5)	64.4 (-0.2)	45.8 (+2.2)	53.3 (+12.3)
CoTTA [47] [CVPR'22]	24.6 (+3.6)	34.8 (+0.6)	44.3 (+0.0)	64.1 (+3.0)	42.2 (+0.0)	64.3 (-0.1)	46.6 (+1.4)	65.2 (+0.4)
SAR [28] [ICLR'23]	26.6 (+1.6)	26.2 (+9.2)	43.2 (+1.1)	54.9 (+9.2)	42.2 (+0.0)	64.2 (+0.0)	40.2 (+7.8)	51.2 (+14.4)
PETAL [1] [CVPR'23]	24.4 (+3.8)	28.0 (+7.4)	36.4 (+7.9)	60.2 (+3.9)	42.2 (+0.0)	64.6 (-0.4)	46.0 (+2.0)	56.3 (+9.3)
ViDA [24] [ICLR'24]	20.7 (+7.5)	27.3 (+8.1)	N/A	N/A	N/A	N/A	N/A	N/A
Continual-MAE [23] [CVPR'24]	12.6 (+15.6)	26.4 (+9.0)	N/A	N/A	N/A	N/A	N/A	N/A
COME [55] [ICLR'25]	26.6 (+1.6)	25.6 (+9.8)	42.6 (+1.7)	61.1 (+3.0)	42.2 (+0.0)	64.2 (+0.0)	45.0 (+3.0)	60.5 (+5.1)
AWS [Proposed]	**10.8 (+17.4)**	**20.4 (+15.0)**	**26.8 (+17.5)**	**50.6 (+13.5)**	**40.7 (+1.5)**	**62.1 (+2.1)**	**30.1 (+17.9)**	**50.2 (+15.4)**
AWS-FS [Proposed]	N/A	N/A	28.2 (+16.1)	52.5 (+11.6)	43.9 (-1.7)	64.3 (-0.1)	31.6 (+16.4)	51.9 (+13.7)

Table 4. **Summary of mean classification error (%) on CIFAR10C and CIFAR100C under CTTA**. Mean (%) denotes the average error rate across 15 target domains. FS denotes the few-shot setup that utilizes a prototype classifier constructed with 30 samples per class.

and 20.4% on CIFAR100C. Compared to the prior state-of-the-art method, we observe performance gains of 2.2% and 5.2%, respectively. Note that we consistently improve on multiple benchmarks, which demonstrates the robustness of our approach and its adaptability to conventional TTA. Additionally, we report N/A for AWS-FS in the source pretrained setting. Since the source classifier is available, the few-shot classifier is not required.

4.3. Results on Self-Supervised Models

ImageNet-to-ImageNetC. In Tab. 3, the experimental results on ImageNetC using each SSL model are represented below source pretrained section. The self-supervised models exhibit relatively low performance on source data compared to the source pretrained model. The absence of source-specific knowledge limits the adaptability of exist-

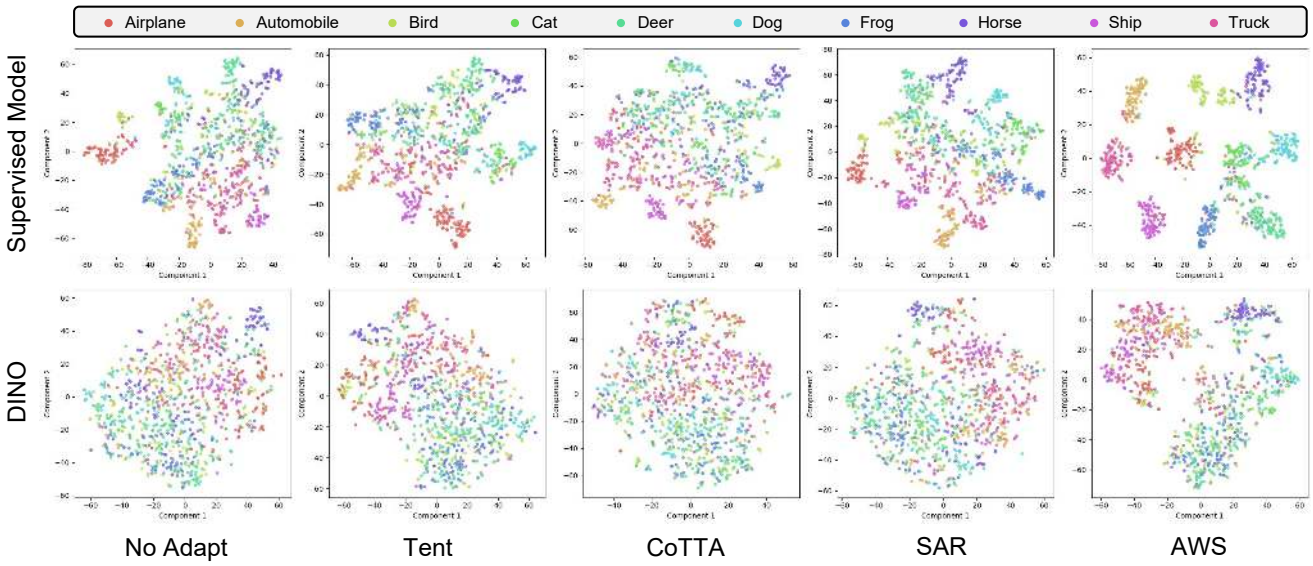


Figure 5. **Feature visualization.** We compare the t-SNE results on CIFAR10C under Gaussian noise. (above) The results of the source pretrained model; (below) the results of the SSL model, DINO.

ing TTA methods, leading to limited performance improvements: 2.6% with DINO, 1.2% with MoCo, and 0.9% with iBOT. In contrast, we observe significant improvements of 16.2%, 7.0%, and 19.9% compared to the baseline. Despite limited initial performance, the proposed method improves the representation and adaptability of SSL models, suggesting its potential for online adaptation.

CIFAR10-to-CIFAR10C & CIFAR100-to-CIFAR100C. In Tab. 4, mean error rates on CIFAR benchmarks for self-supervised models [2, 6, 57] are shown to the right of the source pretrained section. We achieve 26.8%, 40.7%, and 30.1% on CIFAR10C and 50.6%, 62.1%, and 50.2% on CIFAR100C with DINO, MoCo, and iBOT, respectively. As mentioned in Sec. 4.1, we report N/A for adapter-based methods [23, 24] due to the need for additional training parameters. The lower initial accuracy of MoCo probably limits its ability to adapt in the unsupervised setting. Nevertheless, AWS achieves performance gains of 1.5% and 2.1%, representing a meaningful improvement. In conclusion, we observe consistent performance improvements on both source-pretrained and self-supervised models, verifying the effectiveness of the proposed method on standard benchmarks.

Few-Shot Classifier Evaluation. In Tabs. 3 and 4, we report the performance of AWS-FS, where a few-shot classifier is constructed using a subset of the source data containing 30 images per class. We achieve performance improvements of 14.8%, 2.7%, and 18.1% for DINO, MoCo, and iBOT, respectively. Despite the relatively lower gains due to lower source domain performance compared to the full-shot classifiers, reported as AWS; it still achieves consistently significant improvements with respect to conventional methods.

5. Further Analysis

Feature Visualization. We provide t-SNE [42] visualization results to analyze the effect of TTA methods on the distribution of representations in Fig. 5. After adaptation, we extract features from the Gaussian noise corruption in CIFAR10C using both the source pretrained model and the self-supervised model, DINO. Existing approaches are typically designed to preserve the initial representations by updating only normalization layers or employing an EMA model. Consequently, these conservative update strategies demand high initial performance of the source model, leading to dependency on its initial state. In contrast, we observe that the proposed method exhibits improved decision boundaries for both the source pretrained model and the self-supervised model.

Hyperparameter Analysis. The proposed method involves four hyperparameters: k, n, λ_{kd}, and λ_{ml}. We conduct a grid search in Tab. 5 to analyze the sensitivity across all datasets using the source pretrained model. According to Tab. 5a, the best performing configurations of $[k, n]$ are $[1, 5]$ for ImageNetC and CIFAR10C, and $[1, 2]$ for CIFAR100C. Moreover, λ_{kd} and λ_{ml} represents that the best performance is obtained with $\lambda_{kd} = 0.01$ and $\lambda_{ml} = 0.4$. We observe that our method not only exhibits low sensitivity to hyperparameters but also surpasses previous methods across a wide range of hyperparameter settings.

$[k, n]$	IN-C	C10-C	C100-C
$[1, 2]$	39.5	11.5	**20.4**
$[1, 3]$	39.5	11.0	20.5
$[1, 5]$	**39.4**	**10.8**	20.7
$[3, 10]$	40.1	57.2	23.1
$[5, 20]$	40.4	N/A	24.8

(a) **Hyperparameter** $[k, n]$**.**

λ_{kd}	IN-C	C10-C	C100-C
0	40.6	11.1	22.3
0.01	**39.4**	**10.8**	**20.4**
0.02	40.1	11.2	22.5
0.03	41.9	11.7	24.9
0.04	43.6	11.5	26.9

(b) **Hyperparameter** λ_{kd}**.**

λ_{ml}	IN-C	C10-C	C100-C
0	43.8	13.7	25.1
0.1	41.4	12.8	23.9
0.2	40.3	11.7	21.5
0.3	39.8	11.6	20.8
0.4	**39.4**	**10.8**	**20.4**

(c) **Hyperparameter** λ_{ml}**.**

Table 5. **AWS ablation experiments.** We investigate the sensitivity of hyperparameters in proposed method. IN-C, C10-C, and C100-C are ImageNetC, CIFAR10C, and CIFAR100C, respectively.

Method	**Directly test on unseen domains**					**Unseen**
	bri.	contrast	elastic	pixelate	jpeg	Mean↓
No Adapt	26.4	91.4	57.5	38.0	36.2	49.9
Tent	25.8	91.9	57.0	37.2	35.7	49.5
CoTTA	25.3	88.1	55.7	36.4	34.6	48.0
ViDA	**24.6**	68.2	49.8	34.7	34.1	42.3
AWS	24.8	**65.9**	**47.1**	**34.1**	**33.5**	**41.1**

Table 6. **Domain generalization** performance on ImageNetC. Results (%) represent the error rates for unseen domains.

CL	KD	ML	IN-C	C10-C	C100-C
No Adapt [Baseline]			55.8	28.2	35.4
✓			43.4	16.0	27.9
		✓	42.7	21.3	21.8
	✓	✓	41.6	21.3	23.0
✓	✓		43.8	14.1	25.1
✓		✓	40.6	11.2	22.2
✓	✓	✓	**39.4**	**10.8**	**20.4**

Table 7. **Effect of each component**, such as Contrastive Learning (CL), Knowledge Distillation (KD), and Mutual Learning (ML).

Domain Generalization. In Tab. 6, we evaluate the domain generalization performance on ImageNetC. Following ViDA [24], we adapt to 10 corruption types from ImageNetC under the CTTA protocol, and subsequently evaluate performance on the 5 remaining unseen corruption types. We achieves an 8.8% improvement over the source model and surpasses the previous state-of-the-art by 1.2%. These results indicate that the proposed method acquires generalized knowledge and enhances representational capacity during adaptation, thereby improving performance on unseen target domains.

Effectiveness of Individual Components. Tab. 7 presents an ablation study evaluating the contribution of each component in our method, including CL, KD and ML. First, we apply CL to enhance the representational capability of the SSL model and achieve performance improvements of 12.4%, 12.2%, and 7.5% on ImageNetC, CIFAR10C, and CIFAR100C, respectively (row1). These results indicate that applying CL individually can effectively improve adaptation. Second, when KD is introduced to CL, we observe that it mitigates forgetting during adaptation and results in comparable or even lower mean error than using CL alone (row4). Third, adding ML to the combination of CL and KD achieves the best performance across all datasets, demonstrating that ML provides additional benefits for further performance improvement (row6). The ablation study suggests that each component contributes to complementary aspects of the adaptation process.

6. Conclusion

In this paper, we investigate the feasibility of integrating self-supervised models into TTA and explore effective strategies to enhance their adaptability within this scenario. We address the primary challenge of applying self-supervised models to TTA, the absence of a classifier, by proposing a prototype classifier without extra training and cost. Furthermore, we propose AWS, composed of CL, KD, and ML, to effectively leverage the expressive representations of self-supervised models while reducing reliance on source-specific knowledge for more stable adaptation. Extensive experiments demonstrate that our proposed AWS is highly effective not only in the self-supervised setting but also in the conventional supervised setting. Based on these results, we expect this study to contribute to expanding the potential of self-supervised models in TTA and hope that future research will build on these findings.

References

[1] Dhanajit Brahma and Piyush Rai. A probabilistic framework for lifelong test-time adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3582–3591, 2023. 1, 3, 5, 6, 14, 15

[2] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021. 2, 3, 5, 6, 7

[3] Hyuntak Cha, Jaeho Lee, and Jinwoo Shin. Co2l: Contrastive continual learning. In *Proceedings of the IEEE/CVF International conference on computer vision*, pages 9516–9525, 2021. 4

[4] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848, 2017. 1

[5] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020. 3, 4

[6] Xinlei Chen, Saining Xie, and Kaiming He. An empirical study of training self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2021. 2, 3, 5, 6, 7

[7] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2009. 5, 12

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 1, 5, 6, 14, 15

[10] Yossi Gandelsman, Yu Sun, Xinlei Chen, and Alexei A Efros. Test-time training with masked autoencoders. In *Advances in Neural Information Processing Systems*, 2022. 1, 3

[11] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. *Advances in neural information processing systems*, 31, 2018. 5

[12] Jisu Han, Jaemin Na, and Wonjun Hwang. Ranked entropy minimization for continual test-time adaptation. In *Forty-second International Conference on Machine Learning*, 2025. 1

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, 2016. 1

[14] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9729–9738, 2020. 3, 4

[15] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations*, 2018. 5

[16] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, et al. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8340–8349, 2021. 12

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 4

[18] Saihui Hou, Xinyu Pan, Chen Change Loy, Zilei Wang, and Dahua Lin. Learning a unified classifier incrementally via rebalancing. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, 2019. 4, 5

[19] Xu Ji, Joao F Henriques, and Andrea Vedaldi. Invariant information clustering for unsupervised image classification and segmentation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9865–9874, 2019. 5

[20] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 2023. 1

[21] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 1, 5

[22] Jonghyun Lee, Dahuin Jung, Saehyung Lee, Junsung Park, Juhyeon Shin, Uiwon Hwang, and Sungroh Yoon. Entropy is not enough for test-time adaptation: From the perspective of disentangled factors. In *International Conference on Learning Representations*, 2024. 1, 3

[23] Jiaming Liu, Ran Xu, Senqiao Yang, Renrui Zhang, Qizhe Zhang, Zehui Chen, Yandong Guo, and Shanghang Zhang. Continual-mae: Adaptive distribution masked autoencoders for continual test-time adaptation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 3, 5, 6, 7, 13, 14, 15

[24] Jiaming Liu, Senqiao Yang, Peidong Jia, Ming Lu, Yandong Guo, Wei Xue, and Shanghang Zhang. Vida: Homeostatic visual domain adapter for continual test time adaptation. In *International Conference on Learning Representations*, 2024. 1, 3, 5, 6, 7, 8, 13, 14, 15

[25] Yuejiang Liu, Parth Kothari, Bastien Germain van Delft, Baptiste Bellot-Gurlet, Taylor Mordan, and Alexandre Alahi. TTT++: When does self-supervised test-time training fail or thrive? In *Advances in Neural Information Processing Systems*, 2021. 1, 3

[26] Thomas Mensink, Jakob Verbeek, Florent Perronnin, and Gabriela Csurka. Distance-based image classification: Generalizing to new classes at near-zero cost. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2013. 4

[27] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Yaofo Chen, Shijian Zheng, Peilin Zhao, and Mingkui Tan. Efficient test-time model adaptation without forgetting. In *International conference on machine learning*. PMLR, 2022. 3

[28] Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Zhiquan Wen, Yaofo Chen, Peilin Zhao, and Mingkui Tan. Towards stable

test-time adaptation in dynamic wild world. In *Internetional Conference on Learning Representations*, 2023. 1, 3, 5, 6, 12, 14, 15

[29] Shuaicheng Niu, Chunyan Miao, Guohao Chen, Pengcheng Wu, and Peilin Zhao. Test-time model adaptation with only forward passes. In *International Conference on Machine Learning*, 2024. 12

[30] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 3, 4

[31] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 3

[32] David Osowiechi, Gustavo A Vargas Hakim, Mehrdad Noori, Milad Cheraghalikhani, Ali Bahri, Moslem Yazdanpanah, Ismail Ben Ayed, and Christian Desrosiers. Nc-ttt: A noise constrastive approach for test-time training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 3

[33] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, 2017. 4, 5

[34] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do cifar-10 classifiers generalize to cifar-10? *arXiv preprint arXiv:1806.00451*, 2018. 1, 5

[35] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *International conference on machine learning*, pages 5389–5400. PMLR, 2019. 12

[36] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016. 1

[37] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2014. 4

[38] Hidetoshi Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of statistical planning and inference*, 2000. 1, 2

[39] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. *Advances in neural information processing systems*, 30, 2017. 4

[40] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*. PMLR, 2020. 1, 2

[41] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *Advances in neural information processing systems*, 2017. 3

[42] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9 (11), 2008. 7

[43] Raviteja Vemulapalli, Hadi Pouransari, Fartash Faghri, Sachin Mehta, Mehrdad Farajtabar, Mohammad Rastegari, and Oncel Tuzel. Knowledge transfer from vision foundation models for efficient training of small task-specific models. In *International Conference on Machine Learning*, pages 49345–49367. PMLR, 2024. 5

[44] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023. 1

[45] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno A. Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. 1, 2, 3, 5, 6, 12, 14, 15

[46] Haohan Wang, Songwei Ge, Zachary Lipton, and Eric P Xing. Learning robust global representations by penalizing local predictive power. *Advances in neural information processing systems*, 32, 2019. 12

[47] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *Proceedings of Conference on Computer Vision and Pattern Recognition*, 2022. 1, 2, 3, 5, 6, 12, 14, 15

[48] Hongxin Wei, Lei Feng, Xiangyu Chen, and Bo An. Combating noisy labels by agreement: A joint training method with co-regularization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13726–13735, 2020. 5

[49] Yichen Wen, Zhiquan Tan, Kaipeng Zheng, Chuanlong Xie, and Weiran Huang. Provable contrastive continual learning. In *International Conference on Machine Learning*, pages 52819–52838. PMLR, 2024. 4

[50] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler, faster, stronger. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024. 1

[51] Xingrui Yu, Bo Han, Jiangchao Yao, Gang Niu, Ivor Tsang, and Masashi Sugiyama. How does disagreement help generalization against label corruption? In *International conference on machine learning*, pages 7164–7173, 2019. 5

[52] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. In *International Conference on Learning Representations*, 2017. 4

[53] Jiahan Zhang, Qi Wei, Feng Liu, and Lei Feng. Candidate pseudolabel learning: Enhancing vision-language models by prompt tuning with unlabeled data. In *Forty-First International Conference on Machine Learning*, 2024. 4

[54] Marvin Zhang, Sergey Levine, and Chelsea Finn. Memo: Test time robustness via adaptation and augmentation. *Advances in neural information processing systems*, 2022. 1, 3

[55] Qingyang Zhang, Yatao Bian, Xinke Kong, Peilin Zhao, and Changqing Zhang. COME: Test-time adaption by conservatively minimizing entropy. In *International Conference on Learning Representations*, 2025. 3, 5, 6, 14, 15

[56] Yitian Zhang, Xu Ma, Yue Bai, Huan Wang, and Yun Fu. Accessing vision foundation models via imagenet-1k. In *International Conference on Learning Representations*, 2025. 5

[57] Jinghao Zhou, Chen Wei, Huiyu Wang, Wei Shen, Cihang Xie, Alan Yuille, and Tao Kong. ibot: Image bert pre-training with online tokenizer. *International Conference on Learning Representations*, 2022. 2, 3, 5, 6, 7

Appendix

A. Hyperparameters in experiments

In this section, we describe the learning rate configuration used in our experiments on SSL models. For fair comparison, we select the learning rate that yields the lowest mean error within the range defined in [1e-3, 1e-4, 1e-5, 1e-6]$\times\frac{\text{batch size}}{64}$.

1. **DINO**
   - Learning rate (ImageNetC) [**1e-3** (Tent, SAR, CoTTA), **1e-4** (PETAL, AWS), 1e-5, **1e-6** (COME)] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR10C) [**1e-3** (CoTTA, PETAL), **1e-4** (Tent, SAR, COME), **1e-5** (AWS), 1e-6] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR100C) [**1e-3** (SAR, PETAL), **1e-4** (Tent, COME, CoTTA), **1e-5** (AWS), 1e-6] $\times\frac{\text{batch size}}{64}$
2. **MoCo**
   - Learning rate (ImageNetC) [**1e-3** (CoTTA, PETAL), **1e-4** (SAR, COME, AWS), 1e-5, **1e-6** (Tent)] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR10C) [**1e-3** (SAR, COME, CoTTA), 1e-4, **1e-5**(AWS), **1e-6** (Tent, PETAL)] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR100C) [**1e-3** (CoTTA, COME, SAR), 1e-4, **1e-5**(AWS), **1e-6** (Tent, PETAL)] $\times\frac{\text{batch size}}{64}$
3. **iBOT**
   - Learning rate (ImageNetC) [1e-3, **1e-4** (Tent, SAR, CoTTA, PETAL, AWS), 1e-5, **1e-6** (COME)] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR10C) [**1e-3** (SAR, CoTTA), **1e-4** (Tent, COME), 1e-5, **1e-6** (PETAL, AWS)] $\times\frac{\text{batch size}}{64}$
   - Learning rate (CIFAR100C) [**1e-3** (Tent, SAR, CoTTA, PETAL), **1e-4** (COME), 1e-5, **1e-6** (AWS)] $\times\frac{\text{batch size}}{64}$

B. Effect of the Number of Few-Shot Samples

We study the applicability of AWS when the classifier is constructed using a subset of the source data. To this end, we evaluate the effect of varying the number of samples (N) and compare it with the full-shot setting, where the entire dataset is used. Specifically, we compare the mean error rates on ImageNetC using DINO backbone across three different seeds. As shown in Fig. 6, the performance is close to that achieved using the entire dataset as N increases. Notably, $N = 30$ few-shot setting requires only 3% of the source data compared to the full-shot setting. Nevertheless, the performance gap to the full-shot setting remains within 1%. We observe that a classifier constructed with a subset of data can achieve performance close to the full-shot classifier. This observation suggests that our method has the potential to maintain its effectiveness even in more challenging scenarios.

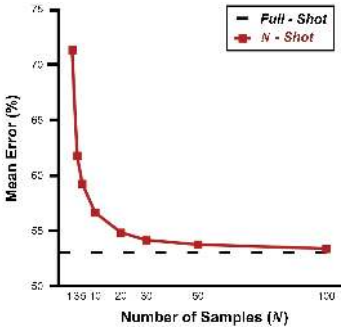


Figure 6. Effect of # of samples

C. Natural Domain Shift Scenarios

In Tab. 8, we provide experiments on natural domain shift datasets to extend the validity of the proposed method in real-world natural changes. We conduct on ImageNet-R [16], V2 [35], Sketch [46] as a target for ImageNet [8] source. We compare the results of our method with Tent [45], CoTTA [47], SAR [28] and FOA [29]. Our method consistently achieves improved performance, demonstrating the effectiveness of AWS.

Method	Source Pretrained			DINO			MoCo			iBOT		
	R	V2	Sketch	R	V2	Sketch	R	V2	Sketch	R	V2	Sketch
No Adapt	59.5	**75.4**	44.9	39.3	56.9	24.3	26.6	52.8	18.3	40.6	59.1	25.0
TENT	63.9	75.2	49.1	39.6	57.0	24.4	26.6	53.2	18.3	41.3	59.1	25.1
CoTTA	63.5	**75.4**	50.0	39.5	57.0	25.5	21.7	52.1	11.2	41.1	59.2	26.8
SAR	63.3	75.1	48.7	39.8	57.0	24.5	**38.0**	**54.2**	19.0	41.4	59.0	25.1
FOA	63.8	**75.4**	**54.4**	42.1	57.1	31.5	20.6	52.6	8.1	44.4	59.0	35.6
AWS (Ours)	**69.3**	**75.4**	**54.4**	**45.0**	**57.5**	**38.0**	35.8	53.5	**25.6**	**48.8**	**59.6**	**40.0**

Table 8. Classification accuracy (%) for ImageNet-to-ImageNet-R/V2/Sketch.

D. Comparison with Adapter-based Methods

We present a comparison of SSL models with ViDA [24] and Continual-MAE [23] in Tab. 9. For self-supervised models, the visual domain adapters are randomly initialized.

Method	Source Pretrained↓	DINO↓	MoCo↓	iBOT↓
ViDA	20.7	43.5	42.1	47.8
Continual-MAE	12.6	44.4	**39.4**	44.1
AWS (Ours)	**10.8**	**26.8**	40.7	**30.1**

Table 9. Classification error rate(%) for CIFAR10-to-CIFAR10C with adapter-based methods.

E. Parameter Update Strategy

We use both SSL model and target model in our framework. The SSL model, trained on large-scale datasets, ensures generalization performance, whereas the target model initialized from it acquires domain-specific knowledge through adaptation. While maintaining the generalized feature representations of the SSL model, we intend to improve the classifier through pseudo labels of the target model that has relatively high accuracy. In AWS, the encoder f_{ssl} is kept frozen during adaptation and the classifier g_{ssl} is updated. Tab. 10 shows EMA updates for f_{ssl} and a fixed classifier g_{ssl}.

Method	Source Pretrained↓	DINO↓	MoCo↓	iBOT↓
g_{ssl} (Frozen)	39.9	**53.0**	73.0	**48.1**
f_{ssl} (Update)	40.0	54.5	70.5	49.7
AWS (Ours)	**39.4**	**53.0**	**69.5**	**48.1**

Table 10. Effect of parameter update strategy for SSL model.

F. Prediction Shifts After Adaptation

Fig. 7 illustrates the change in predictions during adaptation, based on the initial predictions of the source model. The results demonstrate that our method significantly improves predictions on samples initially misclassified by the source model. We interpret this as evidence that our representation learning-based approach is less affected by confirmation bias and more effective at improving the initial model.

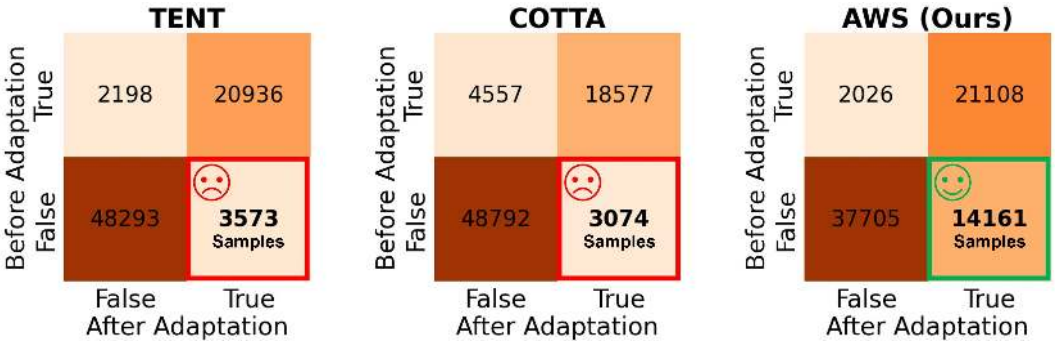


Figure 7. Prediction shifts after adaptation with respect to the source model's initial predictions.

G. Adaptation Time Comparison

We compare the adaptation time for 15 domains from ImageNet-to-ImageNetC in Tab. 11. Our concern about presenting the comparison of adaptation times in the paper is that the adaptation scenarios may depend on the experimental setup, and we emphasize the efficiency of the pretraining time.

Protocol	Source Pretraining Time	Target Adaptation Time (15 domains)	Total Time↓
TTA	1h8m23s×300epochs	8m31s (Tent) – 54m17s (ViDA)	$\leq$ 342h9m17s
SSTTA	1m56s (Few-shot) – 36m15s (Full)	15m30s (Ours)	$\leq$ 51m45s

Table 11. Comparision of adaptation cost.

H. Full-results on CIFAR10-to-CIFAR10C and CIFAR100-to-CIFAR100C

We provide full-results for CIFAR10-to-CIFAR10C and CIFAR100-to-CIFAR100C in Tabs. 12 and 13, respectively. Both include 15 corruption types for each source dataset. The error is measured in an online manner under sequential target domains. We conduct experiments using source pretrained model and self-supervised models such as DINO, MoCo, and iBOT.

Time		t →																
PTM	Method	Gaussian	shot	impulse	defocus	glass	motion	zoom	snow	frost	fog	brightness	contrast	elastic_trans	pixelate	jpeg	Mean↓	Gain
Source Pretrained Acc. 97.1%	No Adapt [9]	60.1	53.2	38.3	19.9	35.5	22.6	18.6	12.1	12.7	22.8	5.3	49.7	23.6	24.7	23.1	28.2	0.0
	Tent [45] [ICLR'21]	57.7	56.3	29.4	16.2	35.3	16.2	12.4	11.0	11.6	14.9	4.7	22.5	15.9	29.1	19.5	23.5	+4.7
	CoTTA [47] [CVPR'22]	58.7	51.3	33.0	20.1	34.8	20.0	15.2	11.1	11.3	18.5	4.0	34.7	18.8	19.0	17.9	24.6	+3.6
	SAR [28] [ICLR'23]	54.1	47.6	38.0	19.9	34.8	22.6	18.6	12.1	12.7	22.8	5.3	39.9	23.6	24.7	23.1	26.6	+1.6
	PETAL [1] [CVPR'23]	59.9	52.3	36.1	20.1	34.7	19.4	14.8	11.5	11.2	17.8	4.4	29.6	17.6	19.2	17.3	24.4	+3.8
	ViDA [24] [ICLR'24]	52.9	47.9	19.4	11.4	31.3	13.3	7.6	7.6	9.9	12.5	**3.8**	26.3	14.4	33.9	18.2	20.7	+7.5
	Continual-MAE [23] [CVPR'24]	30.6	18.9	11.5	10.4	22.5	13.9	9.8	**6.6**	6.5	**8.8**	4.0	**8.5**	12.7	**9.2**	**14.4**	12.6	+15.6
	COME [55] [ICLR'25]	54.3	47.1	36.6	19.9	34.9	22.6	18.6	12.1	12.7	22.8	5.3	40.7	23.4	24.7	23.1	26.6	+1.6
	AWS	**18.7**	**13.1**	**11.0**	**10.1**	**18.8**	**9.7**	**7.2**	7.0	**6.2**	9.5	**3.8**	10.2	**12.4**	9.5	15.4	**10.8**	**+17.4**
DINO Acc. 83.1%	No Adapt [9]	74.8	74.5	67.2	37.0	53.7	34.7	28.8	27.7	32.5	48.6	15.3	44.2	37.8	48.9	39.8	44.3	0.0
	Tent [45] [ICLR'21]	76.2	77.7	68.0	36.3	54.2	34.0	27.2	26.9	30.0	45.7	13.9	38.7	35.3	51.1	36.8	43.5	+0.8
	CoTTA [47] [CVPR'22]	74.8	74.5	67.1	37.1	53.7	34.7	28.9	27.6	32.4	48.5	15.3	44.1	37.7	48.8	39.7	44.3	+0.0
	SAR [28] [ICLR'23]	76.7	78.1	67.4	36.4	53.6	34.0	26.9	26.2	29.0	44.7	13.7	38.1	34.9	51.8	36.2	43.2	+1.1
	PETAL [1] [CVPR'23]	74.6	72.1	64.2	34.4	49.1	33.2	24.5	24.1	24.0	27.7	**11.2**	29.3	**19.4**	36.6	**21.8**	36.4	+7.9
	COME [55] [ICLR'25]	76.4	76.7	66.7	38.2	51.7	34.2	26.5	25.6	28.0	43.7	13.3	35.9	33.9	53.3	35.2	42.6	+1.7
	AWS	**51.3**	**33.9**	**35.8**	**23.3**	**32.5**	**26.0**	**18.3**	**20.4**	**19.6**	**26.8**	12.6	**19.1**	25.1	**27.4**	29.5	**26.8**	**+17.5**
	AWS-FS	53.2	35.6	**35.8**	25.3	33.9	27.8	20.0	21.7	21.0	29.7	13.5	20.5	26.2	28.0	30.3	28.2	+16.1
MoCo Acc. 83.6%	No Adapt [9]	66.7	66.2	64.7	36.3	50.8	39.9	31.5	25.8	32.7	55.9	14.0	29.9	42.3	45.4	**31.0**	42.2	0.0
	Tent [45] [ICLR'21]	67.0	67.3	65.0	36.4	51.4	40.0	31.5	26.5	34.6	56.3	14.2	30.3	42.8	44.7	32.4	42.7	-0.5
	CoTTA [47] [CVPR'22]	66.7	66.2	64.7	36.3	50.8	39.9	31.5	**25.8**	32.7	55.9	14.0	29.9	42.3	45.4	**31.0**	42.2	+0.0
	SAR [28] [ICLR'23]	66.7	66.2	64.7	36.3	50.8	39.9	31.5	**25.8**	32.9	55.6	**13.8**	29.8	42.0	45.2	31.2	42.2	+0.0
	PETAL [1] [CVPR'23]	66.7	66.4	64.8	36.3	51.1	39.5	30.4	26.1	33.8	54.9	14.0	29.2	41.9	44.7	33.0	42.2	+0.0
	COME [55] [ICLR'25]	66.7	66.2	64.7	36.3	50.8	39.9	31.5	**25.8**	33.0	55.9	**13.8**	30.6	42.2	44.9	31.4	42.2	+0.0
	AWS	**66.0**	**64.5**	**62.4**	**34.1**	**49.9**	**37.6**	**27.7**	27.4	**32.5**	**52.2**	14.7	**26.0**	**38.1**	**43.7**	33.6	**40.7**	**+1.5**
	AWS-FS	70.1	68.6	68.2	37.2	53.9	39.2	30.6	29.8	35.1	54.8	17.6	30.1	41.7	45.7	36.0	43.9	-1.7
iBOT Acc. 83.4%	No Adapt [9]	75.8	75.4	70.2	51.1	50.1	43.3	39.5	25.5	29.3	54.7	16.9	48.7	38.8	59.3	42.2	48.0	0.0
	Tent [45] [ICLR'21]	76.0	76.0	70.9	51.5	50.5	41.3	35.2	23.7	27.3	49.1	14.3	40.7	35.7	57.7	37.7	45.8	+2.2
	CoTTA [47] [CVPR'22]	72.1	68.4	68.1	55.9	47.3	48.4	46.9	27.8	24.9	42.6	19.1	50.4	36.0	52.8	37.9	46.6	+1.4
	SAR [28] [ICLR'23]	80.2	81.8	74.2	41.5	48.4	27.0	18.1	19.5	22.3	26.0	14.2	28.2	31.6	57.8	32.7	40.2	+7.8
	PETAL [1] [CVPR'23]	76.7	77.0	71.7	49.8	52.1	42.0	35.9	25.2	29.5	48.3	14.8	39.4	35.6	54.1	37.6	46.0	+2.0
	COME [55] [ICLR'25]	76.4	76.7	71.5	52.1	50.8	38.8	32.3	23.4	27.6	45.0	13.4	37.1	32.9	60.9	35.5	45.0	+3.0
	AWS	**70.1**	**57.6**	**54.4**	**26.7**	**36.4**	**23.7**	**15.1**	**17.4**	**18.5**	**25.9**	**10.4**	**16.5**	**21.5**	29.3	**28.6**	**30.1**	**+17.9**
	AWS-FS	72.2	60.1	53.5	27.7	38.5	25.6	16.5	19.0	20.5	29.0	11.0	17.4	23.3	**29.0**	30.6	31.6	+16.4

Table 12. Full-results for CIFAR10-to-CIFAR10C under CTTA scenario. Mean (%) denotes the average error rate across 15 target domains.

Time		t →																
PTM	Method	Gaussian	shot	impulse	defocus	glass	motion	zoom	snow	frost	fog	brightness	contrast	elastic_trans	pixelate	jpeg	Mean↓	Gain
Source Pretrained Acc. 92.6%	No Adapt [9]	55.0	51.5	26.9	24.0	60.5	29.0	21.4	21.1	25.0	35.2	11.8	34.8	43.2	56.0	35.9	35.4	0.0
	Tent [45] [ICLR'21]	53.0	47.0	24.6	22.3	58.5	26.5	19.0	21.0	23.0	30.1	11.8	25.2	39.0	47.1	33.3	32.1	+3.3
	CoTTA [47] [CVPR'22]	55.0	51.3	25.8	24.1	59.2	28.9	21.4	21.0	24.7	34.9	11.7	31.7	40.4	55.7	35.6	34.8	+0.6
	SAR [28] [ICLR'23]	39.4	31.0	19.8	20.9	43.9	22.6	19.1	20.3	20.2	24.3	11.8	22.3	35.2	32.1	30.1	26.2	+9.2
	PETAL [1] [CVPR'23]	49.2	38.7	24.1	26.3	38.2	25.4	19.4	21.0	19.3	26.6	15.4	31.8	28.3	26.6	29.5	28.0	+7.4
	ViDA [24] [ICLR'24]	50.1	40.7	22.0	21.2	45.2	21.6	16.5	17.9	16.6	25.6	**11.5**	29.0	29.6	34.7	27.1	27.3	+8.1
	Continual-MAE [23] [CVPR'24]	48.6	30.7	18.5	21.3	38.4	22.2	17.5	19.3	18.0	24.8	13.1	27.8	31.4	35.5	29.5	26.4	+9.0
	COME [55] [ICLR'25]	39.5	30.5	19.7	20.7	41.8	22.5	17.2	20.2	17.3	23.7	12.8	22.3	34.7	32.2	29.6	25.6	+9.8
	AWS	**29.0**	**24.0**	**17.2**	**17.8**	**30.5**	**19.3**	**15.7**	**16.7**	**15.7**	**19.2**	11.8	**15.9**	**25.9**	**20.6**	**27.0**	**20.4**	**+15.0**
DINO Acc. 61.5%	No Adapt [9]	82.1	80.6	78.4	57.9	78.2	55.7	49.5	52.0	55.4	69.4	36.3	66.4	62.6	72.6	64.6	64.1	0.0
	Tent [45] [ICLR'21]	80.8	78.8	79.0	58.3	78.1	55.1	47.9	50.0	52.1	67.3	34.2	63.2	60.6	76.0	62.0	62.9	+1.2
	CoTTA [47] [CVPR'22]	82.1	80.6	79.0	57.9	78.3	55.6	49.5	51.9	55.2	69.3	36.4	66.4	62.6	72.7	64.5	64.1	+0.0
	SAR [28] [ICLR'23]	75.3	66.3	71.0	56.1	71.2	51.6	**41.2**	**41.7**	**42.7**	**49.9**	**32.1**	48.0	**49.5**	75.0	**51.4**	54.9	+9.2
	PETAL [1] [CVPR'23]	81.7	79.0	78.0	57.2	70.5	54.1	48.1	50.9	46.6	55.0	41.2	63.1	51.4	69.5	56.2	60.2	+3.9
	COME [55] [ICLR'25]	79.4	75.9	76.4	62.9	75.0	54.8	47.9	46.9	49.0	62.5	33.2	58.3	58.8	76.8	58.2	61.1	+3.0
	AWS	**63.7**	**52.5**	**58.8**	**51.2**	**59.7**	**51.2**	43.9	44.7	43.2	51.6	37.5	**45.7**	52.7	**49.9**	52.9	**50.6**	**+13.5**
	AWS-FS	66.2	54.8	60.2	52.6	62.0	52.9	45.6	47.0	45.3	53.6	39.0	47.9	53.9	52.0	54.6	52.5	+11.6
MoCo Acc. 59.5%	No Adapt [9]	82.8	81.6	83.3	58.5	73.0	60.0	51.7	52.5	56.0	75.0	37.4	56.7	65.6	69.3	59.0	64.2	0.0
	Tent [45] [ICLR'21]	82.9	81.8	83.4	58.5	73.3	60	51.8	53.0	56.9	75.2	37.5	56.7	66.1	69.1	59.5	64.4	-0.2
	CoTTA [47] [CVPR'22]	82.8	81.6	91.6	57.6	72.8	58.9	50.8	52	55.5	74.5	**37.1**	55.9	65.0	69.0	59.9	64.3	-0.1
	SAR [28] [ICLR'23]	82.8	82.6	83.3	58.5	73.0	60.0	51.7	52.5	56.0	74.8	37.3	57.5	65.6	69.1	59.1	64.2	+0.0
	PETAL [1] [CVPR'23]	82.9	81.7	91.1	58.5	73.2	59.7	51.6	52.6	56.2	74.6	37.2	56.2	65.5	68.7	59.5	64.6	-0.4
	COME [55] [ICLR'25]	82.8	81.6	83.3	58.5	73.0	60.0	51.7	52.5	55.9	74.8	37.3	57.5	65.6	69.1	59.0	64.2	+0.0
	AWS	**82.2**	**79.6**	**80.4**	**56.8**	**71.5**	**57.9**	**49.6**	**51.2**	**52.7**	**70.1**	38.1	**53.9**	**62.5**	**65.9**	**58.7**	**62.1**	**+2.1**
	AWS-FS	83.6	81.6	82.3	59.5	72.8	60.3	52.2	54.6	56.2	71.5	41.1	55.1	65.3	67.7	61.1	64.3	-0.1
iBOT Acc. 61.0%	No Adapt [9]	81.3	80.3	81.4	69.2	70.7	62.1	57.1	47.4	48.9	70.5	37.3	71.5	61.4	79.6	66.2	65.6	0.0
	Tent [45] [ICLR'21]	78.8	74.3	76.8	57.7	64.4	45.5	38.2	38.0	38.5	45.9	29.7	42.9	49.5	71.3	47.6	53.3	+12.3
	CoTTA [47] [CVPR'22]	78.9	74.4	78.0	69.3	67.3	62.8	59.5	47.0	45.4	66.2	42.0	76.9	58.3	84.3	67.3	65.2	+0.4
	SAR [28] [ICLR'23]	**74.6**	65.2	73.2	55.7	62.1	**44.8**	38.3	**36.9**	**36.8**	**44.9**	**29.5**	**42.5**	**48.1**	69.4	**45.6**	51.2	+14.4
	PETAL [1] [CVPR'23]	76.5	67.4	71.6	57.3	**59.6**	52.2	45.1	47.4	44.7	54.1	39.5	71.6	48.5	55.0	54.4	56.3	+9.3
	COME [55] [ICLR'25]	79.7	76.2	78.8	66.4	67.7	56.8	52.1	41.6	41.3	56.4	32.5	62.0	54.7	80.5	61.3	60.5	+5.1
	AWS	75.6	**64.9**	**67.0**	**47.6**	60.5	45.9	**37.8**	39.9	39.2	47.7	31.5	44.5	48.2	**48.5**	51.7	**50.2**	**+15.4**
	AWS-FS	76.3	67.4	69.3	49.6	62.5	47.2	39.4	42.3	41.2	49.6	33.7	45.6	50.1	50.3	53.4	51.9	+13.7

Table 13. Full-results for CIFAR100-to-CIFAR100C under CTTA scenario. Mean (%) denotes the mean error rate across 15 target domains.